

Prague University of Economics and Business
Faculty of Informatics and Statistics



A Data Integration Framework for Enterprise Application Security

MASTER'S THESIS

Study program: Applied Data Analytics and Artificial Intelligence

Specialization: Retail and E-Commerce Data Analytics

Author: Bc. Vojtěch Kraus

Supervisor: Ing. Aleš Kotuč

Prague, month YYYY

Acknowledgements

I would like to thank Ing. Aleš Kotuč for his guidance and valuable feedback.

Abstract

To protect their business-critical applications from cybersecurity threats, enterprise security teams rely on numerous tools to detect and resolve vulnerabilities in application source code, configuration, dependencies, CI/CD pipelines, and runtime infrastructure. These tools use different integration patterns, protocols, and data formats, making it difficult to consistently parse, deduplicate, triage, and group their findings. In large environments, additional challenges emerge when linking all findings to the corresponding business applications. This thesis presents a data framework to tackle those challenges, delivering three distinct contributions: (1) a requirements specification for an application security data platform, (2) a reusable and extensible design covering architecture, data model, and medallion-based data pipeline, and (3) a reference implementation built on the Databricks platform. Together, these contributions offer application security teams a foundation for building a robust data platform independent of individual cybersecurity vendors, relieving them of risks usually associated with vendor lock-in.

Keywords

application security, software security, DevSecOps, security integration, data consolidation, medallion architecture, Databricks

Contents

Introduction	11
Motivation	11
Objective	12
Methodology	13
Structure	13
1 Literature Review	15
1.1 Software Development Literature	15
1.1.1 Source Code Management	15
1.1.2 Software Development Lifecycle	15
1.1.3 Application Security Data Standards	15
1.2 Application Security Literature	15
1.2.1 Code Security	16
1.2.2 Software Supply Chain Security	16
1.2.3 Software Lifecycle Security	16
1.2.4 Application Penetration Testing	16
1.3 Data Modeling and Engineering Literature	16
1.3.1 Data Architecture	16
1.3.2 Data Modeling and Pipelines	16
1.4 Related Work and Gap Analysis	16
2 Requirement Analysis	18
2.1 Requirements Methodology	18
2.1.1 Specification Format	18
2.1.2 Requirement Prioritization	18
2.1.3 Test-to-Requirement Linking	18
2.1.4 Traceability Generation	19
2.2 Personas	20
2.2.1 Application Owners	20
2.2.2 Application Developers	20
2.2.3 Security Experts	20
2.2.4 Development and Security Leadership	20
2.3 Data Sources	20
2.3.1 Application Inventory	20
2.3.2 Tools for Software Development and Delivery	20
2.3.3 Runtime Infrastructure Platforms and Security Tools	21
2.3.4 Security Testing Tools	21
2.4 Data Transformations	22
2.4.1 Data Ingestion and Validation	22

2.4.2	Data Normalization	22
2.4.3	Finding Deduplication	22
2.4.4	Business Application Mapping	22
2.4.5	Triage	22
2.5	Data Consumers	22
2.5.1	Remediation Tracking Systems	22
2.5.2	Application Security Posture Management (ASPM)	22
2.5.3	SOAR and SIEM	22
2.6	Non-Functional Requirements	22
2.6.1	Data Quality	22
2.6.2	Extensibility	23
2.6.3	Scalability	23
2.6.4	Performance	23
2.6.5	Security	23
2.6.6	Maintainability	23
2.7	Requirement Specification	23
2.7.1	Data Ingestion	23
2.7.2	Data Processing	23
2.7.3	Data Serving	23
2.7.4	Non-Functional Requirements	23
3	Solution Architecture	24
3.1	Architecture Context	24
3.2	Architecture Principles and Constraints	24
3.3	Technology Selection	24
3.3.1	Data Engineering Platform	24
3.3.2	Pipeline Framework	24
3.3.3	Connectors	25
3.4	High-Level Solution Architecture	26
3.5	Component Design	26
3.5.1	Data Ingestion Tier	26
3.5.2	Data Processing Tier	26
3.5.3	Data Serving Tier	27
3.6	Integration Architecture	27
3.6.1	Connector Strategy	27
3.7	Extensibility Architecture	27
3.8	Security Architecture	28
4	Pipeline and Model Design	29
4.1	Pipeline Overview	29
4.2	Bronze Layer	29
4.2.1	Transformations	29
4.2.2	Schema	30
4.3	Silver Layer	30

4.3.1	Transformations	30
4.3.2	Schema	30
4.4	Gold Layer	31
4.4.1	Transformations	31
4.4.2	Schema	31
5	Implementation	32
5.1	Environment Setup	32
5.2	Connector Implementation	32
5.3	Pipeline Implementation	32
5.4	Pipeline Orchestration	32
5.5	Data Serving	32
5.6	Testing and Validation	32
	Conclusion	33
	Thesis Outcomes and Contributions	33
	Limitations	33
	Future Work	33
	References	34
A	Generative AI Use Disclosure	36
A.1	Text Content	36
A.2	Diagrams and Figures	36
A.3	Source Code	37
A.4	Requirements Specification	37

List of Figures

3.1	Component Design	26
4.1	Medallion Layer Design	29

List of Tables

List of source codes

- 2.1 Requirement Marker Example 19
- 2.2 DLT Expectation Example 19
- 3.1 GitHub Security Connector Implementation 25

Introduction

Motivation

To protect business-critical applications from cybersecurity threats, security teams need to use a large set of application security tools: Static Application Security Testing (SAST) scanners analyze source code for vulnerabilities, Software Composition Analysis (SCA) tools detect issues in software dependencies, Dynamic Application Security Testing (DAST) solutions scan running applications for exploitable weaknesses, and secret scanning tools identify exposed credentials in code.

Further detection mechanisms need to be applied to software integration and deployment pipelines that are prone to their own classes of threats. Increasingly, security teams employ mechanisms that identify problems before they even reach the version control system, either in the form of IDE plugins or as security context tools integrated into AI coding assistants like Github Copilot or Claude Code.

In large enterprises, there may be dozens of such vulnerability detection tools, each producing findings through different Application Programming Interfaces (APIs), data formats, and integration patterns ([Anderson, 2020](#); [Stallings & Brown, 2024](#)).

As application security architect in a large organization managing tens of thousands of code repositories and thousands of developers, I have witnessed firsthand how challenging it is to keep such a complex environment secure. Most notably:

- Different detection tools of the same class capture different results as they employ different fundamental scanning paradigms
- The same vulnerability may be reported by multiple tools, requiring careful deduplication
- Findings lack context regarding business criticality of the software project or its threat exposure, both of which are critical determinants of how urgently a vulnerability needs to be remediated
- TODO more

I argue that the challenge of consolidating application security data is, at its core, a data engineering problem. While the industry is addressing this with commercial products categorized as Application Security Posture Management (ASPM) that offer dashboards and aggregated views, these solutions are proprietary, expensive, and inflexible. They introduce vendor lock-in and limit the ability of security teams to customize data pipelines, apply their own machine learning models, or integrate with internal systems on their own terms.

The most notable open-source alternative, DefectDojo, takes a different approach: it is

a Django-based web application designed primarily as a vulnerability management interface. However, it was not built with a data-first architecture in mind. Its scaling capabilities are limited by the constraints of a traditional relational database backend, and its data ingestion model relies predominantly on push-based integrations or manual report uploads. Pull-based connectors that actively fetch data from security tool APIs are available only in the commercial DefectDojo Pro offering, further fragmenting the landscape between open and proprietary solutions.

What is missing is a vendor-agnostic, data-first framework that treats application security data as a first-class data engineering problem. One that provides a reusable architecture for ingestion, normalization, deduplication, and serving of security findings at enterprise scale. This thesis aims to fill that gap.

Objective

The main goal of this thesis is to design and implement a vendor-agnostic data integration framework for enterprise application security. The framework consolidates security findings from heterogeneous tools into a unified data platform, enabling consistent normalization, deduplication, triage, and correlation of vulnerabilities across business applications.

To achieve this goal, I define the following subgoals:

1. **Literature review** ([Chapter 1](#)): Survey existing literature across three domains: software development practices, application security, and data engineering. Identify the theoretical foundations and the gap that motivates this work.
2. **Requirement specification** ([Chapter 2](#)): Analyze the application security domain to identify personas, data sources, required data transformations, data consumers, and non-functional requirements. Produce a requirements specification for the framework, with the full specification included in the appendix.
3. **Architecture** ([Chapter 3](#)): Propose a reusable and extensible architecture that satisfies both analytical and transactional use cases.
4. **Pipeline and model design** ([Chapter 4](#)): Design the data model and processing pipeline following the medallion architecture pattern ([Strenghtolt, 2025](#)).
5. **Implementation and validation** ([Chapter 5](#)): Use AI-assisted coding to build a reference implementation on the Databricks platform ([Lee et al., 2024](#)) and validate it against the requirements specification through automated testing with requirement traceability.

Methodology

This thesis follows a design-science research methodology ([Hevner et al., 2004](#); [Peppers et al., 2007](#)), structured into three phases.

In the first phase, I conduct a **literature review** of academic and industry sources across three domains: software development practices and lifecycle, application security tools and standards, and data engineering approaches, including data architecture and pipeline design. This review establishes the theoretical foundation for the framework and includes vendor documentation for the technologies used in the implementation.

In the second phase, I perform a **requirements analysis and design**. I analyze the application security landscape to identify the key personas, data sources, required transformations, and non-functional requirements. Requirements are prioritized using the MoSCoW method. Based on the requirements, I propose an architecture and design a data model with a medallion-based processing pipeline.

In the third phase, I build a **reference implementation** on the Databricks platform to demonstrate feasibility of the proposed design. I validate the implementation through automated tests with requirement traceability, mapping individual tests to requirements, and through data quality expectations enforced by [Delta Live Tables \(DLT\)](#).

The thesis delivers three distinct contributions:

1. A **requirements specification** for an application security data platform that other organizations can adapt to their own needs.
2. A **reusable design** covering architecture, data model, and pipeline that is not tied to a specific platform or vendor.
3. A **reference implementation** on Databricks that proves the feasibility of the design and serves as a starting point for production deployments.

Structure

This thesis is organized as follows. [Chapter 1](#) surveys the relevant literature on software development, application security, and data engineering, and identifies the research gap. [Chapter 2](#) defines the personas, data sources, transformations, consumers, and non-functional requirements, culminating in a prioritized requirements specification. [Chapter 3](#) presents the platform architecture, including technology selection and component design. [Chapter 4](#) details the medallion-based data model and pipeline transformations across the bronze, silver, and gold layers. [Chapter 5](#) describes the reference implementation on Databricks, covering connector development, pipeline orchestration, and testing. The Conclusion evaluates the outcomes against the objectives defined above, discusses limitations,

and suggests directions for future work.

1. Literature Review

1.1 Software Development Literature

What process/activities are we securing with appsec?

Books like Code Complete, Clean Code, Pragmatic Programmer...

1.1.1 Source Code Management

git book, software engineering at Google, ...

1.1.2 Software Development Lifecycle

Traditional vs devops and CI/CD pipelines, list books on each

1.1.3 Application Security Data Standards

Standard data formats for vulnerabilities like Common Vulnerabilities and Exposures (CVE)

SARIF, CycloneDX, OCSF

Citing test ([Anderson, 2020](#)), ([Strenghtolt, 2025](#)),

ASPM mentioned for the first time, ASPM mentioned second time.

API

1.2 Application Security Literature

In this section we focus on literature which explains the domain in which we're solving identified problems.

Books on general appsec

1.2.1 Code Security

1.2.2 Software Supply Chain Security

1.2.3 Software Lifecycle Security

1.2.4 Application Penetration Testing

Good source of wisdom for defensive teams - let's defend against what the attackers are doing

1.3 Data Modeling and Engineering Literature

What tools do we have to achieve thesis goals?

1.3.1 Data Architecture

DWH

Lake

Lakehouse etc

1.3.2 Data Modeling and Pipelines

Data Modeling with Snowflake

DAMA-DMBOK

Medallion Architecture

1.4 Related Work and Gap Analysis

Security data integration - who wrote about it

An industry cybersecurity data model exists but only for SOC events, not for appsec modeling

Does DefectDojo have any useful models?

ASPM exists but not data oriented and not very customizable

2. Requirement Analysis

The goal of this chapter is to conduct a requirement analysis and produce a structured requirements specification which we will later use to design the data framework.

2.1 Requirements Methodology

Requirements are gathered through application security domain analysis, drawing from my personal experience, consultations with my professional network, and by having researched literature and industry reports in [Chapter 1](#).

This thesis adopts a test-bound approach to requirements traceability. Rather than maintaining disconnected requirement documents that are prone to becoming stale, implementation status is derived directly from test execution results.

2.1.1 Specification Format

Each requirement is assigned a unique identifier following the pattern REQ-AREA-NNN, where:

- AREA indicates the functional domain (ING for ingestion, PROC for processing, SRV for serving)
- NNN is a sequential number within that domain

2.1.2 Requirement Prioritization

Requirements are documented with priority using the MoSCoW method (Must Have, Should Have, Could Have, Won't Have).

2.1.3 Test-to-Requirement Linking

The framework employs two complementary verification mechanisms aligned with Databricks development patterns.

Unit and Integration Tests

Python modules containing connector logic and transformation functions are tested using pytest with custom requirement markers. [Source code 2.1](#) demonstrates how tests reference requirement IDs.

Source code 2.1

Requirement Marker Example

```
1 @pytest.mark.requirement("REQ-ING-002")
2 def test_sonarqube_pagination():
3     """Validate pagination handling for large result sets."""
4     ...
```

Data Quality Expectations

Data quality requirements are verified using Delta Live Tables expectations, which execute during pipeline runs. [Source code 2.2](#) shows how expectations map to requirements via code comments.

Source code 2.2

DLT Expectation Example

```
1 @dlt.table
2 @dlt.expect_or_fail("valid_severity", "severity IN ('critical', 'high', 'medium', 'low')")
3 @dlt.expect("has_finding_id", "finding_id IS NOT NULL")
4 def silver_findings():
5     # REQ-NFR-001: Data Quality Validation
6     return transform_findings(dlt.read("bronze_raw"))
```

Expectation results are captured in the pipeline event log and reflected in requirement status.

2.1.4 Traceability Generation

A generator script produces the traceability matrix by:

1. Scanning test files for `@pytest.mark.requirement` markers
2. Parsing DLT pipeline code for expectation comments referencing requirement IDs
3. Collecting pytest results and DLT pipeline event logs
4. Merging with requirement descriptions from specification

Appsec vendors, products and their capabilities, APIs and their maturity/completeness

Capture data formats and integration patterns of the tools

2.2 Personas

2.2.1 Application Owners

2.2.2 Application Developers

2.2.3 Security Experts

Architects and Engineers assigned to application teams

Including security compliance / auditors

2.2.4 Development and Security Leadership

Dashboards, metrics, trends, quantified risk

2.3 Data Sources

2.3.1 Application Inventory

Start with looking at things from a business application perspective

Business application inventory + business impact assessment

source systems:

2.3.2 Tools for Software Development and Delivery

important data sources for inventory

start with SCM and issue tracking tools

Github, Gitlab, Bitbucket, Azure DevOps

Jira, Azure DevOps, Github and Gitlab issues

CI/CD pipelines - Jenkins, Github Actions, Azure DevOps

2.3.3 Runtime Infrastructure Platforms and Security Tools

We need infra portrayed from app angle - what does the app run on, how is it secure, whats the security context

AWS - EC2, serverless Azure GCP

Also how secure is it (infra VMDR like Qualys, Wiz...) can impact risk of app vulns if they run on vulnerable infra too

How to get data from Kubernetes, Docker even if on prem

2.3.4 Security Testing Tools

Primary vulnerability discovery tools (secret scanning, , , ...)

Secret Scanning Tools

Static Application Security Testing (SAST) Tools

SAST

Software Composition Analysis (SCA) Tools

SCA

Dynamic Application Security Testing (DAST) Tools

DAST

Penetration Testing

Need a way to upload manual findings, e.g. PDF report from external pentesters

2.4 Data Transformations

2.4.1 Data Ingestion and Validation

Filtering?

2.4.2 Data Normalization

Standard format for each object and finding type

2.4.3 Finding Deduplication

When 2 tools find the same thing, link it, don't create 2 issues

2.4.4 Business Application Mapping

2.4.5 Triage

2.5 Data Consumers

2.5.1 Remediation Tracking Systems

Jira, ServiceNow

2.5.2 Application Security Posture Management (ASPM)

Can pull from data platform already cleaned data instead of from each source separately

2.5.3 SOAR and SIEM

2.6 Non-Functional Requirements

2.6.1 Data Quality

Data validations and transformations on input are critical

2.6.2 Extensibility

Important feature of the whole framework, especially on data source integrations and ML models

2.6.3 Scalability

it's intended for large enterprises, 10k's of repos +

2.6.4 Performance

Some use cases require low latency

2.6.5 Security

Obviously, with this use case

2.6.6 Maintainability

SaaS preferred over manual maintenance - Databricks is ideal

All source code versioned in git and following best practices

2.7 Requirement Specification

2.7.1 Data Ingestion

2.7.2 Data Processing

2.7.3 Data Serving

2.7.4 Non-Functional Requirements

3. Solution Architecture

generic concepts and open industry standards instead of proprietary concepts/technology unless necessary

Needs to support both OLTP/OLAP use cases on data serving tier

3.1 Architecture Context

It will be used to pull data from data sources (those are naturally out of scope)

It needs to be ready to support an application security management dashboard with a combination of OLTP and OLAP use cases (dashboard application also out of scope).

3.2 Architecture Principles and Constraints

3.3 Technology Selection

What I'm choosing and why, what were the alternatives and reasons not to choose them

3.3.1 Data Engineering Platform

Choosing **Databricks** over Snowflake, Fabric, BigQuery

Contains a strong data governance framework - **Unity Catalog**, good for security non-functional requirement

3.3.2 Pipeline Framework

I will use **DLT**, a "data engineering pipeline framework running on top of Delta Lake that combines incremental ingestion, streamlined ETL, and automated data quality processes such as expectations" ([Lee et al., 2024](#)).

The advantages of **DLT** are...

3.3.3 Connectors

(general plan, more details in Integration Architecture)

dlt, Fivetran?, Databricks Partners?

data load tool (dlt) is utilized to ingest data from security tools into the Bronze layer. Source code 3.1 shows an example implementation of a connector for GitHub security data.

Source code 3.1

GitHub Security Connector Implementation

```

1  import dlt
2  from dlt.sources.helpers.rest_client import paginate
3
4  @dlt.source
5  def github_security_source(org: str, token: str):
6      """Extract security data from GitHub organization."""
7
8      @dlt.resource(write_disposition="merge", primary_key="id")
9      def repos():
10         """Repository inventory"""
11         yield from paginate(f"/orgs/{org}/repos")
12
13     @dlt.resource(write_disposition="merge", primary_key="number")
14     def code_scanning_alerts():
15         """GitHub Code Scanning (SAST) alerts"""
16         yield from paginate(f"/orgs/{org}/code-scanning/alerts")
17
18     @dlt.resource(write_disposition="merge", primary_key="number")
19     def dependabot_alerts():
20         """Dependabot (SCA) vulnerability alerts"""
21         yield from paginate(f"/orgs/{org}/dependabot/alerts")
22
23     return repos, code_scanning_alerts, dependabot_alerts

```

Unity Catalog

Delta Lake

Python vs Scala vs SQL

3.4 High-Level Solution Architecture

3.5 Component Design

See Figure 3.1

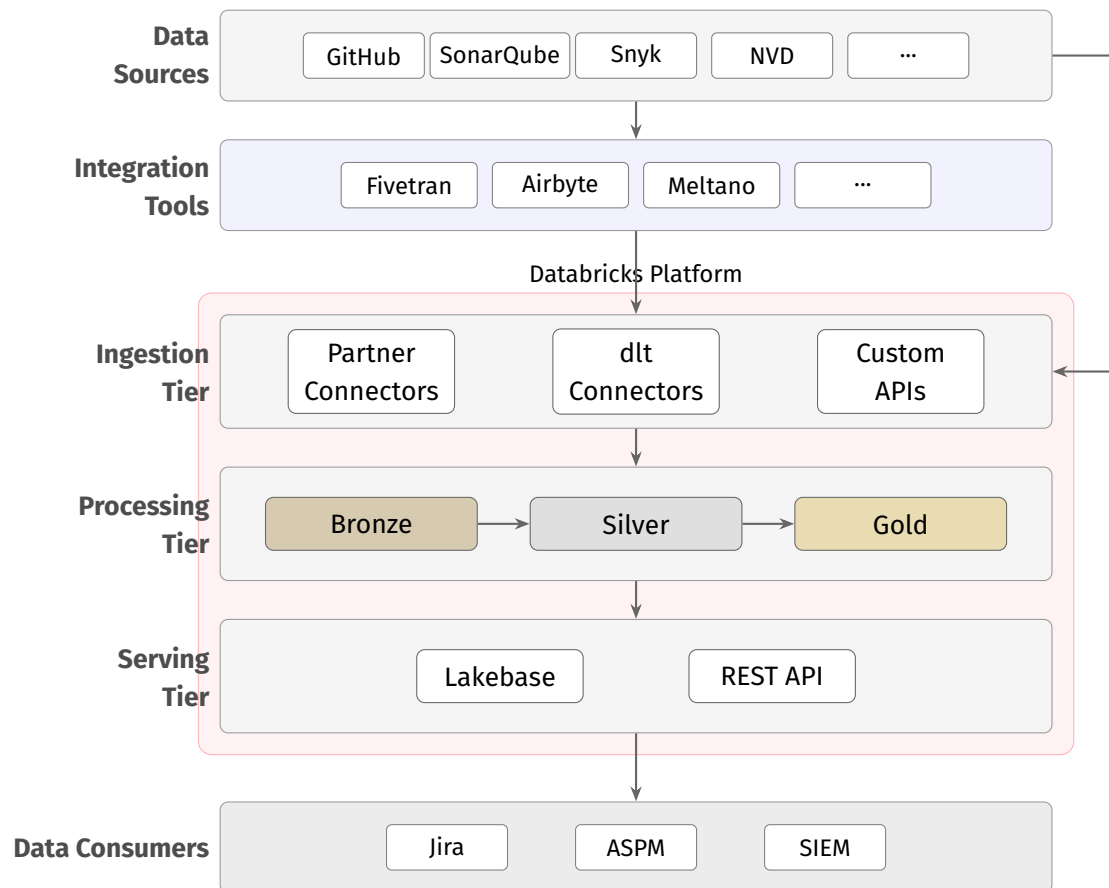


Figure 3.1

Component Design

3.5.1 Data Ingestion Tier

The various connectors

3.5.2 Data Processing Tier

Pipeline Architecture

Medallion architecture, see Chapter 4

DLT for defining transformations

Pipeline Orchestration

Databricks Workflows, don't think my workflows will be complex enough to use Airflow DAGs

3.5.3 Data Serving Tier

Lakebase

3.6 Integration Architecture

3.6.1 Connector Strategy

Different patterns and tools optimal for different data sources

As a service: Fivetran if it satisfies requirements (easiest to do, no need to reinvent the wheel)

Reuse existing: e.g. from DefectDojo <https://github.com/DefectDojo/django-DefectDojo/tree/master/doj>

Databricks Partner

dlt as default tool for custom connectors to REST APIs

Application Inventory

ServiceNow CMDB

Call ServiceNow API vs build ServiceNow app

3.7 Extensibility Architecture

Add new integrations via API

3.8 Security Architecture

4. Pipeline and Model Design

4.1 Pipeline Overview

Each medallion layer is implemented as a schema within a single Unity Catalog.

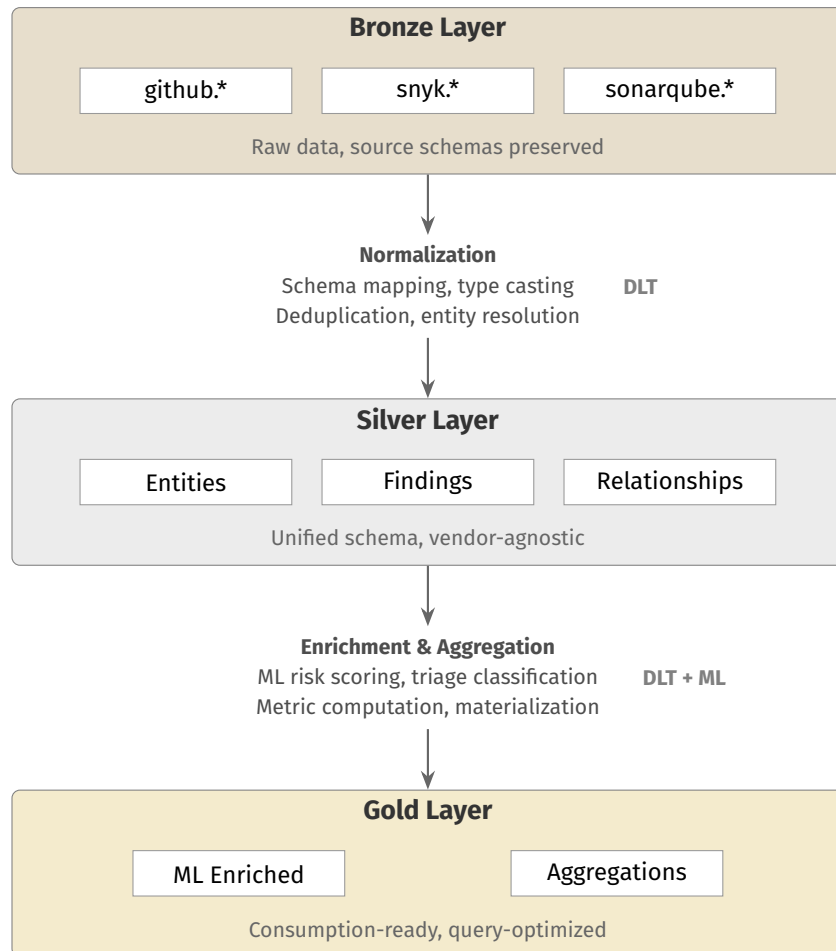


Figure 4.1
Medallion Layer Design

4.2 Bronze Layer

4.2.1 Transformations

Data source request filtering / configuration

Ingestion

4.2.2 Schema

by systems

4.3 Silver Layer

4.3.1 Transformations

Data Quality

Error handling

Normalization

Deduplication

4.3.2 Schema

Vendor-agnostic, unified for each entity and finding

Entities

Implemented as dimensions

Findings

Implemented as facts?

Relationships

(entity-to-entity, finding-to-entity)

4.4 Gold Layer

4.4.1 Transformations

ML Enrichment

Risk scoring

Triage

4.4.2 Schema

Aggregations

Metrics, counts, summaries...

Sliced by severity, app, owner, team, etc (requirement of grouping)

SLAs, trends, MTTR - what matters to stakeholders per stakeholder analysis

Read-Optimized Views

For dashboard queries, materialized, cached

5. Implementation

5.1 Environment Setup

Databricks on AWS

Workspace config

Users, groups, catalogs, schemas

5.2 Connector Implementation

Which connector which pattern and technology

What's the rate limiting on each? This is big data potentially

Authentication, pagination etc, don't forget to mention

5.3 Pipeline Implementation

5.4 Pipeline Orchestration

5.5 Data Serving

5.6 Testing and Validation

Conclusion

This thesis presented a data integration framework for enterprise application security...

Thesis Outcomes and Contributions

Evaluation against objectives from Introduction

Which requirements from Ch2 were met / partially met / not met

What did I deliver extra - ML models beyond requirements?

Limitations

What was not scoped / validated, constraints (license fees for commercial tools that don't give free dev trials)

What was difficult about implementing the design? What could not be implemented in the technology that I selected?

Future Work

More data sources / connectors, based on my provided blueprint

ML model improvements and fine-tuning

Potentials as an open source project

References

- Anderson, R. (2020). *Security engineering: A guide to building dependable distributed systems* (3rd ed.). John Wiley & Sons. (Cit. on pp. [11](#), [15](#)).
- Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, 28(1), 75–105 (cit. on p. [13](#)).
- Lee, D., Wentling, T., Haines, S., & Babu, P. (2024). *Delta Lake: The definitive guide: Modern data lakehouse architectures with data lakes*. O'Reilly Media. (Cit. on pp. [12](#), [24](#)).
- Peppers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems*, 24(3), 45–77 (cit. on p. [13](#)).
- Stallings, W., & Brown, L. (2024). *Computer security: Principles and practice* (5th ed.). Pearson. (Cit. on p. [11](#)).
- Strengtholt, P. (2025). *Building medallion architectures: Designing with Delta Lake and Spark*. O'Reilly Media. (Cit. on pp. [12](#), [15](#)).

Appendices

A. Generative AI Use Disclosure

This appendix contains disclosures on all usage of generative AI tools for this thesis.

A.1 Text Content

All textual content in this thesis was drafted and written exclusively by the author in **Overleaf Online LaTeX editor** with logged history of incremental manual edits.

For language revision, Overleaf embedded **AI Assist tool** with **OpenAI GPT** language model was used with following custom prompt:

```
Article usage rules:
- Omit "the" before: plural nouns used generally, abstract concepts,
  technology names, proper nouns, section references
- Omit "a/an" when: introducing concepts already understood from context,
  in technical definitions, after "using/via/through"
- Keep articles only when: specific instance is referenced, first
  introduction of a countable noun, or omission creates ambiguity
- Examples of preferred style:
  - "Bronze layer stores raw data" (not "The Bronze layer stores the raw
    data")
  - "Figure 3 shows architecture" (not "Figure 3 shows the architecture")
  - "Data flows through pipeline" (not "The data flows through the
    pipeline")
```

A.2 Diagrams and Figures

All TikZ diagrams were generated by Anthropic **Claude Opus 4.5** model through iterative prompting. The author provided specifications, reviewed outputs, and iterated until the outcome met structural and visual requirements.

An example prompt for Figure 3.1:

```
Create a TikZ pipeline architecture diagram with vertical flow:

Components:
1. Data Sources row: GitHub, SonarQube, Snyk, NVD, ...
2. External Fivetran box (outside Databricks boundary)
3. Ingestion tier: Fivetran Connector, dlt Connectors, Custom APIs
```

4. Processing tier: Bronze, Silver, Gold layers (colored)
5. Serving tier: Lakebase, REST API
6. Data Consumers row: Jira, ASPM, SIEM

Wrap Ingestion/Processing/Serving in Databricks Platform boundary.
Use arrows showing data flow between tiers.
Style: Soft colors, rounded corners, compact layout.

A.3 Source Code

Implementation code was written by the author with assistance from:

- **Claude Code** (Anthropic) with Opus 4.5 model for code generation, review, and debugging.
- **Databricks AI Assistant** for in-editor code debugging and SQL optimization.

All generated code was reviewed, tested, and modified by the author before inclusion.

A.4 Requirements Specification

Requirements in Chapter ?? follow a test-centric traceability approach. Tests reference requirement IDs via pytest markers, and a generator script produces the traceability matrix by scanning test results. Implementation status is derived from test execution rather than manual tracking.